THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 02 OF 04

Distributed Rate Limiter

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS
READ · REFLECT · PRACTICE

02



YOUR STUDY COMPANION

A clear, step-by-step system design guide

Purpose: Prepare for a Solution Architect system design discussion. **Approach:** Understand the problem first. Add one component at a time. For every decision, explain **what**, **why**, and **what could go wrong**. **Design chosen:** API Gateway + shared Redis + Token Bucket + atomic update.

NAVIGATION

Inside this guide

01	1. Start with the problem, not the tools
02	2. The final design in one picture
03	3. Understand the Token Bucket using real numbers
04	4. Build the solution one decision at a time
05	5. One request from start to finish
06	6. Other algorithms — when would we choose them instead?
07	7. What happens when Redis or a gateway fails?
08	8. What changes when we have multiple regions?
09	9. Scaling, capacity, security, and monitoring
10	10. Very small technical appendix (optional on first reading)
11	11. Decisions and trade-offs — quick recap
12	12. The interview explanation in one minute
13	Official references

CHAPTER

1. Start with the problem, not the tools

Imagine we run a payment API used by many merchants. One merchant sends far more requests than allowed. We want to protect our backend without blocking normal users.

Business rule (example): Each merchant can send requests at an average rate of **2 per second**, and can send a short burst of **up to 5 at once**.

A rate limiter answers one question for each incoming request:

> **Should this request enter the system now, or should we reject it and ask the client to retry later?**

The rate limiter does **not** process a payment. It only protects the API from too much traffic.

Why is it called *distributed*?

Our API runs on three gateway instances behind a load balancer. A merchant's requests can arrive at any instance.



The broken design: Give each gateway its own counter. If the intended allowance is 5 requests, each gateway might allow 5. The merchant could then send **15** requests across the three gateways. The limit is no longer shared.

What we need: All gateways must consult the **same logical quota state** for a given merchant.

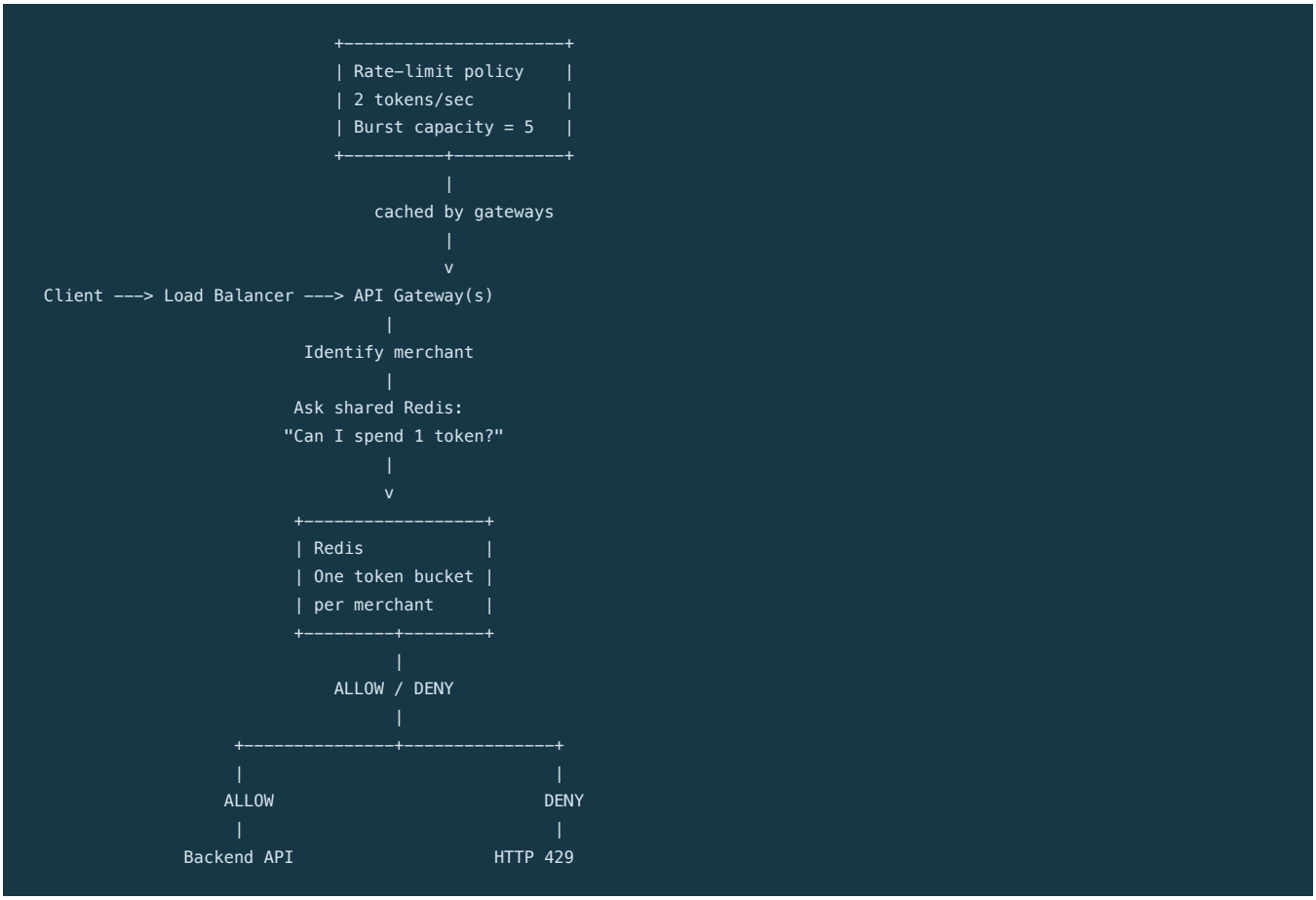
Requirement summary

Must support	Meaning
Per-merchant rate limits	Merchant A and Merchant B should have separate allowances.
Burst allowance	A short spike is acceptable, but sustained overload is not.
Multiple gateways	A caller must not bypass the limit by reaching another gateway.
Fast decisions	Check happens before the backend call, so it should add little latency.
Graceful denial	Return HTTP `429 Too Many Requests`, optionally with `Retry-After`.
Failure behavior	Decide in advance what happens if the limiter's storage is unavailable.

Assumptions: These are example requirements for teaching, not American Express production figures. Exact limits, SLA, and regional scope would come from the actual interview prompt.

CHAPTER

2. The final design in one picture



Read this diagram from top to bottom:

1. The **API gateway** receives requests before the application.
2. It identifies the merchant using **verified authentication** (not a user-supplied merchant name).
3. It looks up that merchant's **rate-limit policy** in its local policy cache.
4. It asks **Redis**, the shared state store, whether the request can spend a token.
5. Redis performs **one atomic check-and-update** and says **allow** or **deny**.
6. Only allowed requests reach the backend.

Why not put the limiter inside every application service? A gateway can reject requests *before* they consume application resources. Services can still apply extra limits for expensive operations. The gateway is the common first line of enforcement.

CHAPTER

3. Understand the Token Bucket using real numbers

Think of a bucket that holds up to **5 tokens**.

- Every incoming request needs **1 token**.
- The bucket starts **full: 5 tokens**.
- It refills at **2 tokens per second**.
- It never holds more than **5 tokens**.

- If there is not enough token balance, **reject** the request.

Two important settings:

Setting	In this example	Controls
Capacity	5 tokens	Maximum immediate burst
Refill rate	2 tokens/second	Long-run request rate

Walk through the requests

Assume all gateways use **one shared bucket** for Merchant A. Every request costs one token.

Time	Request arrives at	Tokens before decision	Result	Tokens afterward
`0.00s`	Gateway A	5	Allow	4
`0.00s`	Gateway B	4	Allow	3
`0.00s`	Gateway C	3	Allow	2
`0.00s`	Gateway A	2	Allow	1
`0.00s`	Gateway B	1	Allow	0
`0.00s`	Gateway C	0	**Deny (429)**	0
`0.25s`	Gateway A	0.5 (refilled)	**Deny (429)**	0.5
`0.50s`	Gateway B	1 (refilled)	Allow	0
`1.00s`	Gateway C	1 (refilled)	Allow	0

What this proves: It does not matter which gateway receives a request. They all spend from the same bucket, so the six simultaneous requests do **not** produce six separate allowances.

Why fractional tokens? Refilling 2 tokens every second means 0.5 token arrives in 0.25 seconds. The limiter can track partial refill, but the request is only allowed once at least 1 full token is available.

The refill formula, explained simply

```
elapsed_seconds = now - last_checked_time
new_tokens = min(capacity, old_tokens + elapsed_seconds * refill_rate)

if new_tokens >= request_cost:
    allow the request and subtract its cost
else:
    deny the request
```

- `min(capacity, ...)` prevents unlimited accumulation during idle periods.
- `elapsed_seconds * refill_rate` calculates how many tokens returned.
- The current count **must be updated atomically**, or simultaneous gateways could each spend the same token.

Important distinction: Token Bucket means an *average rate with permitted bursts*. It does **not** mean “at most two requests in every possible one-second interval.” If the interviewer needs a strict rolling-window guarantee, choose a different algorithm.

CHAPTER

4. Build the solution one decision at a time

Decision 1 — Where should we check the limit?

Choose: API Gateway.

Why: The gateway sees incoming API traffic before forwarding to the application. It can reject excessive traffic without loading payment services or databases.

Alternative: A limit within an individual service is useful when that service needs a *second*, specialized limit, such as a maximum number of expensive fraud checks running simultaneously.

Decision 2 — How do we identify whose quota is used?

Choose: trusted `merchant_id` from authentication, plus a route group.

Examples of independent keys:

```
ratelimit:merchant-42:payments-read
ratelimit:merchant-42:payments-write
ratelimit:merchant-73:payments-read
```

Merchant 42 does not consume Merchant 73's allowance. Read-heavy endpoints can have a different policy from write endpoints.

Why not only IP address? Many real users can share an IP address (office networks, mobile providers), and a caller can change IPs. IP-based rules are still useful for unauthenticated abuse protection. Do not trust `merchant_id` supplied directly by the caller without validating it.

Decision 3 — Where should the quota state live?

Choose: shared Redis.

A limiter needs small, frequently updated state: **current tokens and the last refill timestamp**. Redis is well suited to this in-memory access pattern.

Why not local memory? Each gateway would give its own allowance.

Why not a relational database as the default? A database could be made correct, but updating a row on *every request* introduces avoidable contention and additional load for an ephemeral counter. We can still use a durable database to store **policy definitions**, which do not change on every request.

Start simple: One Redis primary plus a replica can serve a small deployment. Use Redis Cluster when benchmarks or availability requirements justify sharding. Cluster is **not automatically required** just because there are multiple gateways.

Decision 4 — How do we prevent two gateways spending the same token?

Imagine Merchant A has only **1 token** left:

```
Gateway A: READ 1 token --> thinks ALLOW
Gateway B: READ 1 token --> thinks ALLOW
Gateway A: WRITE 0
Gateway B: WRITE 0

Outcome: 2 requests allowed using 1 available token (WRONG)
```

This is a **race condition**: two operations see the same old state before either update is complete.

Choose: one atomic Redis script. The script must do all of this as one unit: read state → refill → decide → deduct → save.

Redis runs a Lua script atomically, so another request cannot interleave its update while this script is executing. Keep it short to avoid blocking other Redis work. Redis's official scripting documentation confirms this behavior.

```
Gateway A ---->
                [Redis atomic operation] ----> Allow A, remaining = 0
Gateway B ---->
                [Redis atomic operation] ----> Deny B, remaining = 0
```

Concept — Atomicity: Other callers cannot observe or modify an operation halfway through. Atomicity here applies to the Redis check-and-update, **not** to the entire later payment transaction.

Decision 5 — What do we send back to the client?

If Redis says **allow**, forward the request and let the application produce its normal response.

If Redis says **deny**, do not call the backend:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 1

{"error":"RATE_LIMIT_EXCEEDED","message":"Please retry later"}
```

`Retry-After: 1` means wait at least one second. The server should calculate this from the refill rate and round up to whole seconds. HTTP `429` and the optional `Retry-After` header are defined by **RFC 6585**.

Decision 6 — How do we avoid stale state and waste?

Choose: an expiry time (TTL) on inactive bucket keys.

A merchant who stops sending requests should not keep an empty/unused counter in Redis forever. An expired bucket can be treated as full when the merchant returns **only after enough idle time has passed to refill it completely**.

Example: capacity 5 and refill rate 2/s → an empty bucket needs **2.5 seconds** to refill. Set idle TTL **longer than this** with safety margin. This is an eviction policy for temporary state, not a business data-retention rule.

Decision 7 — How do we change limits by customer tier?

Choose: durable policy store + cached policy in gateways.

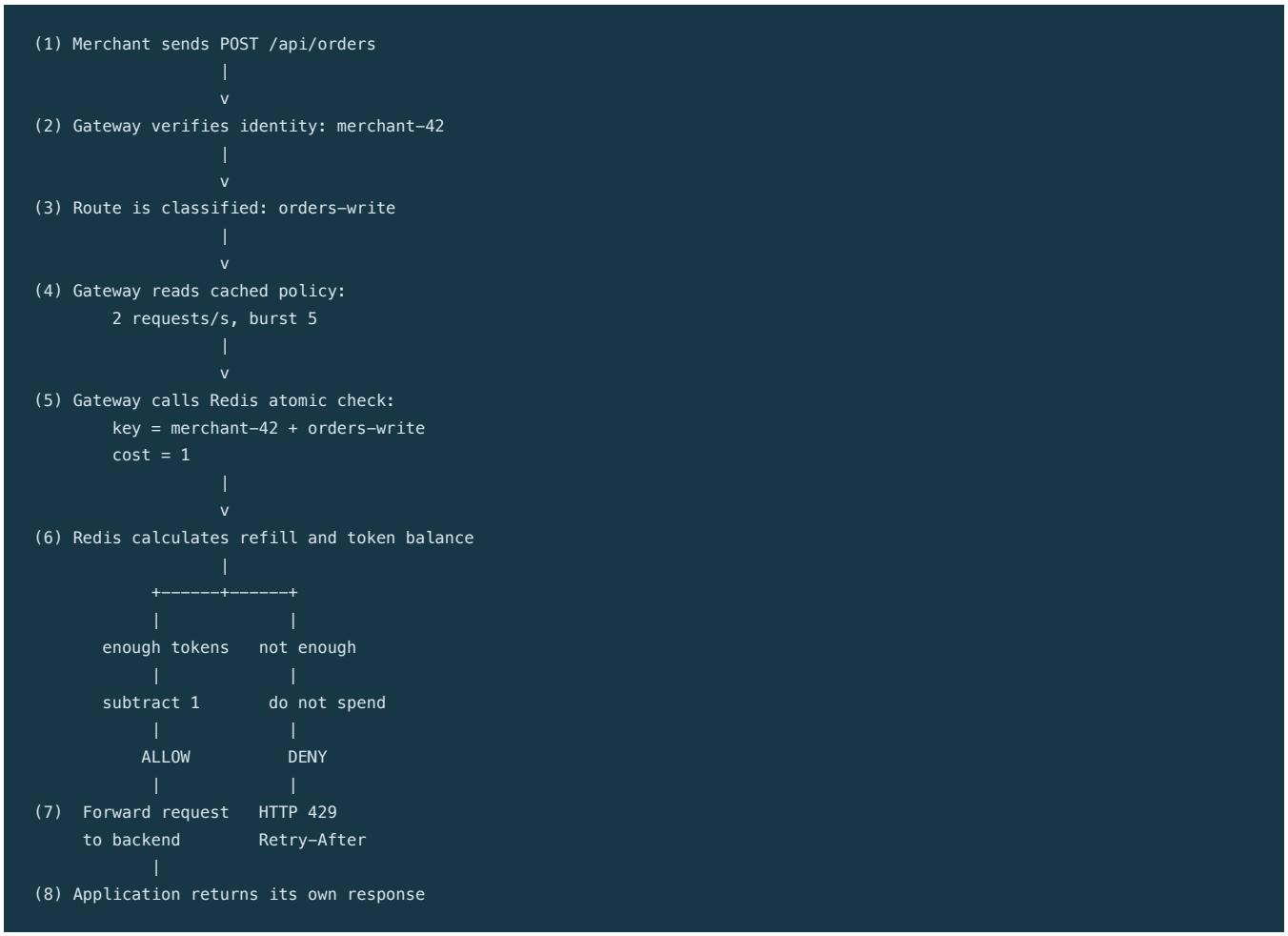
```
Policy database / Admin API
          |
      versioned update
          |
   Gateway policy caches
          |
   Redis checks use the current settings
```

The policy is **configuration** (e.g., premium merchant = 20/s). Tokens are **runtime state** (e.g., 3.5 currently available). Do not query a durable configuration database on every request.

Important: Changing capacity or refill rate can make old token state inconsistent with the new policy. Clamp tokens to the new capacity; plan how policy changes take effect. Avoid silently resetting all buckets to full during a version rollout if that matters.

CHAPTER

5. One request from start to finish



Notice what is *not* in the critical path: policy administration, log aggregation, analytics databases, and dashboards. Those should not slow every API request.

CHAPTER

6. Other algorithms — when would we choose them instead?

The **business rule determines the algorithm**. These are not interchangeable.

Algorithm	Simple idea	Best for	Weak point
Fixed window	Count requests in a calendar window, e.g., each minute	Very simple quotas	Boundary spike: a user may send many requests at the end of one minute and start of the next.
Sliding-window log	Keep timestamps of requests within the last 60 seconds	Strict “N requests in *any* 60 seconds”	More memory and maintenance per key.
Sliding-window counter	Blend counts from the current and	Low-memory approximation of a	Approximate, not exact.

	previous windows	rolling limit	
Token bucket	Spend tokens, refill at a steady rate	**Allow bursts but cap long-run rate**	Does not enforce an exact rolling-window maximum.
Leaky bucket / paced queue	Smooth work at a set departure rate	Smooth downstream processing	May delay requests and require queue management.

Interview rule: If asked “Why Token Bucket instead of Fixed Window?”, answer: **We need to allow small legitimate bursts without letting requests stay above the sustained rate. Fixed Window has sharp boundaries that cause unfair spikes.**

CHAPTER

7. What happens when Redis or a gateway fails?

A working design is not enough. The architect must define **failure behavior**.

Case A — One gateway crashes

The load balancer routes new requests to other healthy gateways. The shared quota remains in Redis. **No quota reset** occurs just because a gateway dies.

Case B — Redis is slow or unavailable

The gateway cannot know the latest shared token balance. Two common policies:

Mode	Behavior	Example rationale
Fail-open	Let requests through, perhaps with a small local emergency limit	Availability is more important than exact temporary enforcement for a low-risk API.
Fail-closed	Reject requests while the limiter is unavailable	Protect a costly or sensitive endpoint from uncontrolled traffic.

No single mode is always correct. Document the choice per endpoint, give Redis calls a short timeout, and monitor how often the fallback is used.

Case C — Redis primary fails over to a replica

The new primary might not have the latest token deductions. Redis Cluster uses asynchronous replication, so **some acknowledged limiter updates can be lost** during failover. This can briefly admit more requests than planned.

Meaning: Atomic checks on the current Redis primary do **not** imply permanently exact quota accounting. For ordinary protective rate limiting, a small temporary overshoot may be acceptable. For billing or a strict contractual quota, this architecture alone is not enough.

Case D — Thousands of requests from one merchant

Even with Redis Cluster, **one hot merchant key** lives on one shard and can saturate it.

Possible solutions, in increasing complexity:

1. Block obvious abuse earlier at the WAF / edge.
2. Isolate a high-volume tenant on a dedicated quota partition.
3. Give gateways **bounded token leases** from an authoritative coordinator (fewer central checks, but some possible overshoot).

Do **not** split one strict 5-token quota into unrestricted counters on multiple shards: that repeats the original problem.

CHAPTER

8. What changes when we have multiple regions?

Suppose the same merchant's requests reach the US and Europe simultaneously.



If both regions independently start with **5 tokens**, the merchant might use **10 tokens**. Regional replicas do not magically create a strict worldwide allowance.

Three practical designs:

Requirement	Approach	Trade-off
Low latency, approximate global limit	Allocate fixed budgets to regions (e.g., US 3, EU 2)	Uneven demand wastes unused regional capacity unless rebalanced.
Strict per-merchant decisions	Assign the merchant one **home/authoritative region** for its checks	Other regions pay cross-region latency; loss of home region affects enforcement.
Fast regional checks with bounded overshoot	Lease limited token blocks to regions/gateways	More coordination and non-zero potential overshoot.

Architecture point to say in an interview: “We must choose whether we prefer strict global consistency or low-latency regional availability. Independent regional Redis stores cannot provide both.”

CHAPTER

9. Scaling, capacity, security, and monitoring

Capacity: calculate the right thing

If we receive **20,000 requests/second** and make **one rate-limit check per request**, Redis must handle approximately **20,000 limiter decisions per second**. Rejected requests still need a decision. If each request checks three separate rules, work can approach **60,000 checks per second**.

These are **illustrative calculations**, not Redis throughput guarantees. Actual capacity depends on hardware, network, scripts, key distribution, and failover headroom. Benchmark the workload and add shards only when measurements justify them.

Security essentials

- Use a verified identity for authenticated quotas; separately protect pre-auth endpoints with IP and edge-based safeguards.

- Keep Redis private; use network controls, TLS where supported, restricted credentials, and key rotation.
- Do not put access tokens, card numbers, or personal data into Redis key names.
- Rate limiting is **not** authentication, authorization, fraud detection, or payment idempotency.

Monitor these signals

- **Allowed vs. denied requests:** Are we blocking the expected traffic?
- **Redis check latency and errors:** Is the limiter slowing every request?
- **Redis CPU, memory, evictions, and hot keys:** Are shards overloaded or losing state?
- **Fail-open / fail-closed counts:** Are we exposing APIs or rejecting valid traffic during outages?
- **Policy-version consistency:** Are all gateways enforcing the same current policy?

CHAPTER

10. Very small technical appendix (optional on first reading)

What exactly is stored in Redis?

```
Key:     rate:merchant-42:orders-write
Value:   tokens = 2.5
         last_refill_ms = 1730000000000  (example timestamp)
TTL:     longer than full-refill time, with safety margin
```

The limiter needs **two values per bucket**. It does not need to store every request or a complete customer profile.

Atomic check-and-update pseudocode

The gateway sends `key`, `capacity`, `refill_rate`, and `cost` to **one Redis script**.

```
ATOMIC_CHECK(key, capacity, refill_rate, cost):
    now = Redis server time
    state = load bucket(key) or start with a full bucket

    elapsed = max(0, now - state.last_refill)
    available = min(capacity, state.tokens + elapsed * refill_rate)

    if available >= cost:
        available = available - cost
        decision = ALLOW
        retry_after = 0
    else:
        decision = DENY
        retry_after = (cost - available) / refill_rate

    save bucket(available, now) and idle TTL
    return decision, retry_after
```

Why one script, not separate Redis calls? The check and write must be atomic. If two gateways read the same old value in separate operations, both might allow a request that should only have been allowed once.

Clock detail: Use Redis server time so different gateway clocks do not change refill calculations. Host clocks still need synchronization, especially around failover.

HTTP example

```
GET /api/transactions HTTP/1.1
Authorization: Bearer <redacted>
```

When over the quota:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 1
Content-Type: application/json

{"code":"RATE_LIMITED","message":"Try again later"}
```

For production, the implementation must additionally validate policy values, handle script reloads, and define the behavior for multiple rate-limit rules on one request.

CHAPTER

11. Decisions and trade-offs — quick recap

Question	Answer	Why
Where do we reject?	Gateway	Protect the backend early.
Which algorithm?	Token Bucket	Permits bursts while controlling sustained rate.
Where is state?	Shared Redis	All gateway replicas see one logical merchant bucket.
Why atomic Lua?	Avoid race conditions	Two gateways cannot spend the same token.
Why TTL?	Remove idle buckets	Prevent unbounded state growth.
How to scale?	Measure first; shard Redis if needed	Independent keys can be distributed across primaries.
What about Redis failure?	Explicit fail-open/closed policy	Availability and protection have different trade-offs.
Is the quota exact globally?	Not with separate regional buckets	Strict global quotas require a single authority or coordination.

CHAPTER

12. The interview explanation in one minute

> “I would enforce limits at the API gateway so excessive requests are rejected before reaching the backend. Since gateways run on multiple instances, local counters would be incorrect. I would use Redis as shared state, with one token bucket per authenticated merchant and route group. A token bucket fits our requirement because we want a sustained rate with a small allowed burst. Each

> gateway executes one short atomic Redis script to refill, check, and spend tokens. Allowed requests go to the backend; denied requests receive HTTP 429 and a retry delay. I would add TTLs for idle buckets, monitor Redis latency and hot keys, and define explicit fail-open or fail-closed behavior for Redis outages. For multi-region traffic, I would state that independent counters are not globally exact, then choose regional budgets or a single authoritative region according to the required consistency."

CHAPTER

Official references

1. **Redis:** Rate limiter use case — why shared state is needed and supported algorithm types.
2. **Redis:** Scripting with Lua — atomic execution and script caching behavior.
3. **Redis:** Redis Cluster specification — sharding, failover, and asynchronous replication.
4. **Redis:** TIME command — server time used for refill calculations.
5. **IETF RFC 6585:** HTTP 429 Too Many Requests — response status and optional `Retry-After`.